Criterion A: Planning

Application Name: FocusGrid

Client: Escher Lang (Co-founder, CodeNest)

Advisor: Rick Waren (Flutter Developer)

Problem Definition

One recurring and significant issue that I have found in my conversations with Escher Lang, co-founder of CodeNest, a technology startup at the beginning of its formation, is a lack of cohesion in terms of managing his teams' time and tasks. CodeNest was using a combination of several tools for managing its developers' and designers' activities — one application for maintaining a calendar, another one for writing a to-do list, and the third one for leaving sticky notes.

As per Escher, the biggest disadvantage of such a workflow is the lack of clearness in terms of scheduling. Team members had conflicts in their schedules, missed deadlines due to the difference between applications, and unclear priorities. Changing a project milestone required updating three different platforms, thus resulting in some delays. Escher admitted that he spends more than twenty minutes of his morning every day aligning his schedule and to-do list before starting working productively — several hours weekly for a five-member team (see Appendix 1).

There were no applications available on the market which could offer both calendar-awareness and customizable reminders. The existing applications either offered only task lists and did not have calendar feature or provided very complicated project management systems which would take too long for our startup company to integrate.

Having learned about these challenges, I suggested to develop FocusGrid – an application which combines task scheduling, calendar features, and reminders. It provides users with a bird's eye view of the current schedule, allows attaching reminders to any calendar date and highlighting free periods of time in order to plan tasks proactively.

To make sure that we have an appropriate architecture of our application, I turned to Rick Waren who is a professional developer of Flutter and whom I have met due to professional network. His advice allowed me to choose Flutter as our development framework, Hive as our local database, and Google Calendar API as integration layer with CodeNest ecosystem.

Solution Rationale

After understanding my client's concerns from the first interview (Appendix 1), I outlined the key components and solutions

Component	Solution	Reason / Rationale
Cross-platform compatibility	Built with Flutter (Dart)	Escher's team employs both Android and iOS devices. The Flutter framework supports a single codebase which natively runs on both platforms, thus reducing the effort of maintaining two applications
Unified task and calendar view	TableCalendar widget with an integrated reminder overlay	By merging the calendar view and task view on a single page, users will get immediate information about their schedule without having to switch apps, which is the very issue of Escher's double booking problem.
Data persistence and offline access	Hive NoSQL local database	All reminder information is stored locally on the device. Hive is a small-sized key-value store database which operates offline and doesn't require any cloud hosting fees.
Google Calendar synchronisation	Google Calendar API (OAuth 2.0)	CodeNest employees are currently using Google Workspace. Integration of FocusGrid reminders into Google Calendar allows for synchronization of calendars on all devices and among coworkers without double-data entry.
Smart scheduling suggestions	Free-slot detection algorithm run on stored reminders	The application analyzes current reminders and discovers periods of time that are free and then shows them to the user to allow Escher's team to plan ahead.
Intuitive, accessible UI	Dark neon-themed GUI with a consistent widget hierarchy	Distinctive visual appearance allows the user to focus on tasks, and the bright colors on the dark background make it possible to find important buttons easily, especially during late night meetings mentioned by Escher.
Notification alerts	flutter_local_notifications package	Push notifications serve as reminders of upcoming tasks even if the application is closed.

Table 1.1: Solution Rationale

Criteria for Success

Each success criterion below is written to be objectively measurable, so that it can be directly tested against the finished product in Criterion D.

1. App Setup and Layout
    a. The launch screen displays the current month's calendar view within 2 seconds of the app opening with the navigation bar visible at all times.
    b. The dark neon UI theme is applied consistently across 100% of screens and widgets in the app.
2. Reminder Functionality
    a. The user can create a reminder by completing 6 fields: name, date, time, colour label (chosen from a palette of at least 8 colours), sound (chosen from at least 3 tones), and priority level (High / Medium / Low).
    b. The user can add an optional note of up to 250 characters to any reminder.
    c. If any of the 4 mandatory fields (name, date, time, priority) is left blank, the form blocks submission and displays an error message within 1 second.
    d. Editing a saved reminder updates it in both the calendar view and the list view within 1 second, with no duplicate entries created.
    e. Deleting a reminder removes it from local storage immediately, so it no longer appears in any view of the app.
3. Calendar View
    a. The full-month interactive calendar (TableCalendar widget) correctly displays all days of the selected month, matching the device's system calendar with 100% accuracy.
    b. Any day with one or more reminders shows a coloured dot indicator matching that reminder's colour label.
    c. Tapping a date filters the reminder list to show only that date's reminders in under 1 second.
4. Reminder List View
    a. All reminders due within the next 30 days appear in the list view, sorted chronologically by date and time.
    b. Each list item displays 5 data points: reminder name, date, time, priority, and colour label.
    c. Tapping a reminder opens its full detail screen within 1 second, from which it can be edited or deleted.
5. Smart Duggestions Features
    a. The app provides users with insightful time management and task completion advice that users may rely on to boost productivity
    b. The smart suggestions should cycle through to provide multiple points of advice.
6. Notification Service
    a. A push notification is delivered within 60 seconds of each reminder's scheduled time, with 100% delivery reliability while notification permissions are granted.
    b. Each notification body displays the reminder name and, where present, up to the first 100 characters of its notes.